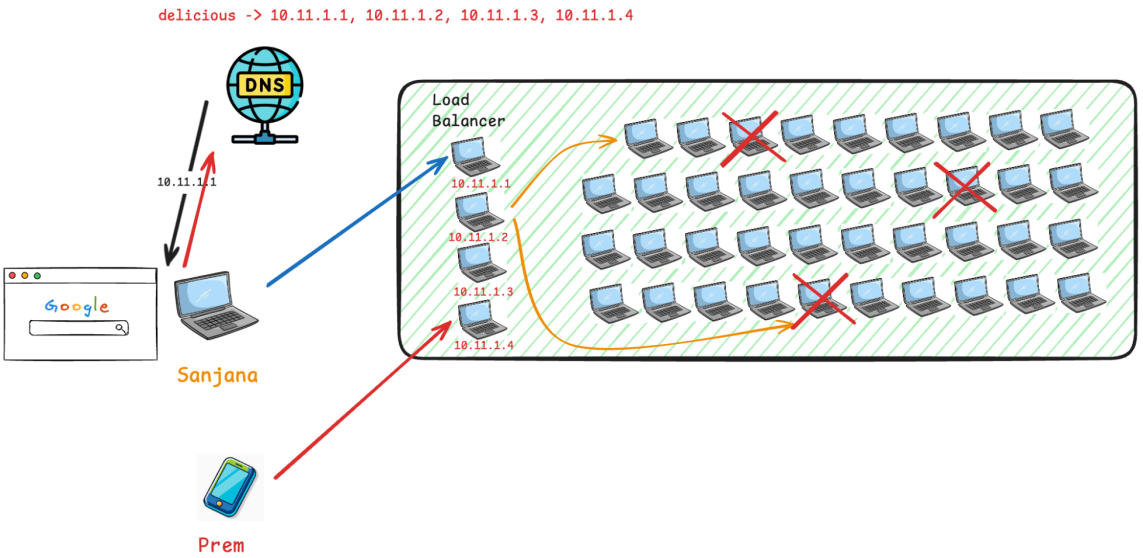
*For any suggestions or feedback regarding these notes,*
*please contact [Pragy Agarwal]*

# Load Balancing

Purpose

1. provide a unified view of the entire backend system to the end user
   *because the end user doesn't are which exact server handles their request*

2. **distribute the load (requests/data) equally across the other app/db servers**




### Q: Which server should a particular request be sent to?

To understand this, we first must understand how the data is distributed across the various servers.
Because otherwise, we will end up sending the requests to servers which don't contain the appropriate data *(Sanjana's request get sent to a server which contains Ashok's data)*

# How to store the data?

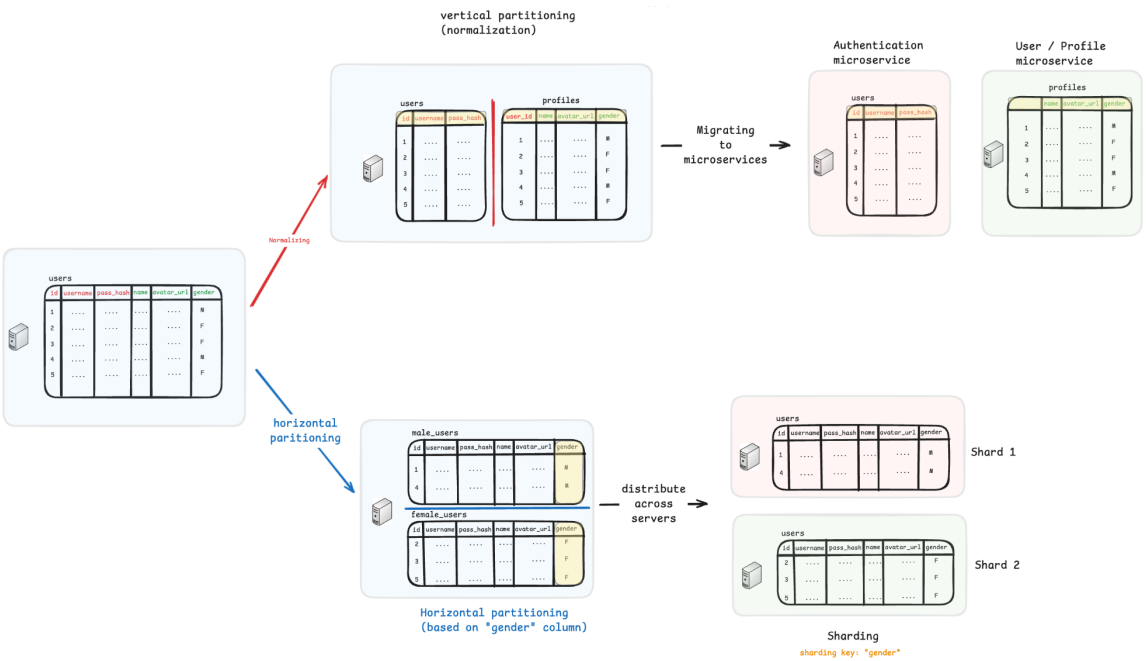
### Q: Can we store all the data on 1 server?

***No.*** We got multiple laptops because we were running out of resources (disk space / CPU / ..)

### Q: Should we split the data randomly?

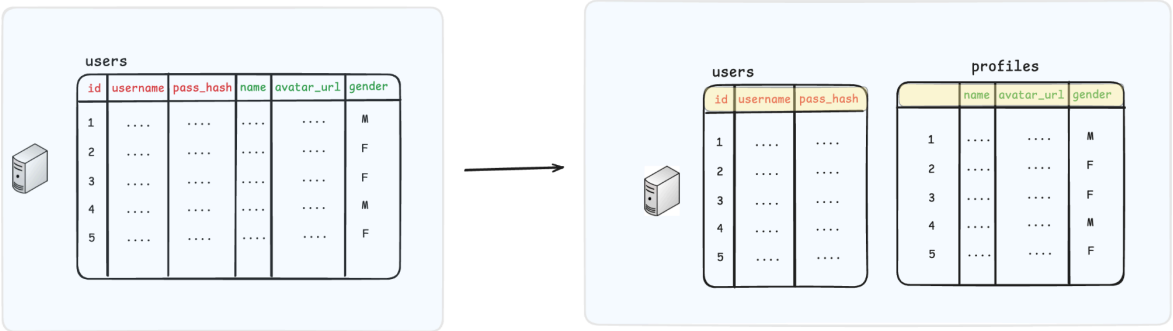
How will we retrieve it?
The data storage has to follow some logic - that can be repeated.
We should be able to find out at any time, which data is stored in which server.

# Data Sharding




# Vertical Partitioning (Normalization)

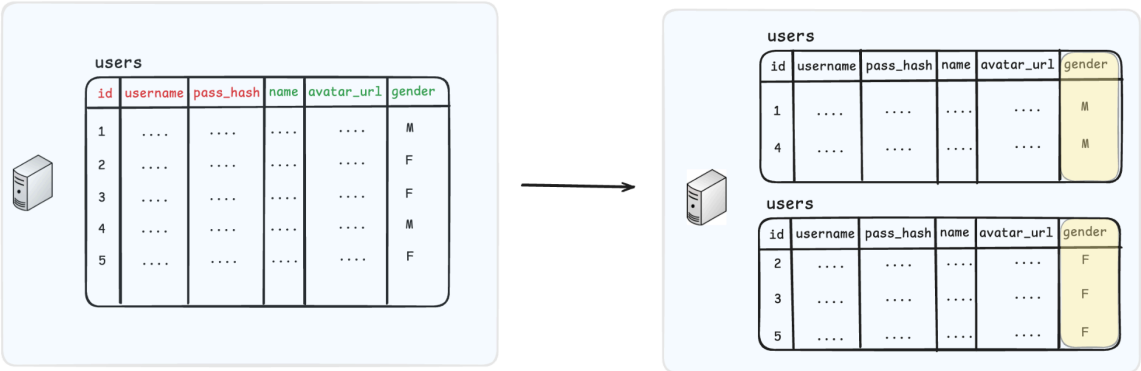
why? Because you want to normalize it




# Horizontal Partitioning

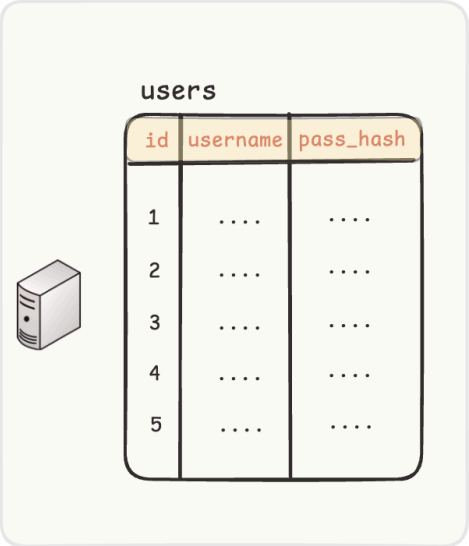
why?

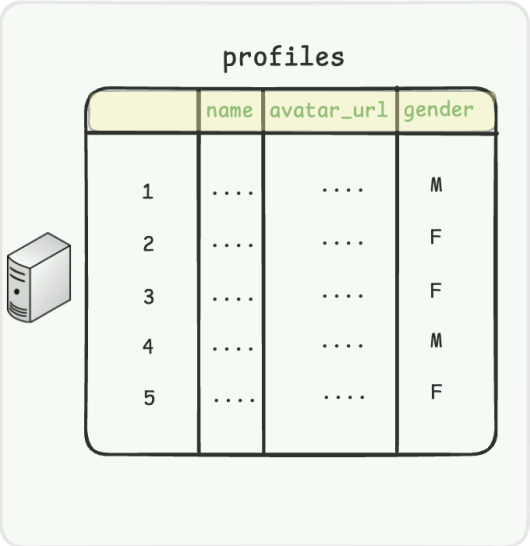
- improve indexing performance
- support multi-tenancy (prevent data cross-communication between clients)




# Vertical Partitioning (across servers)

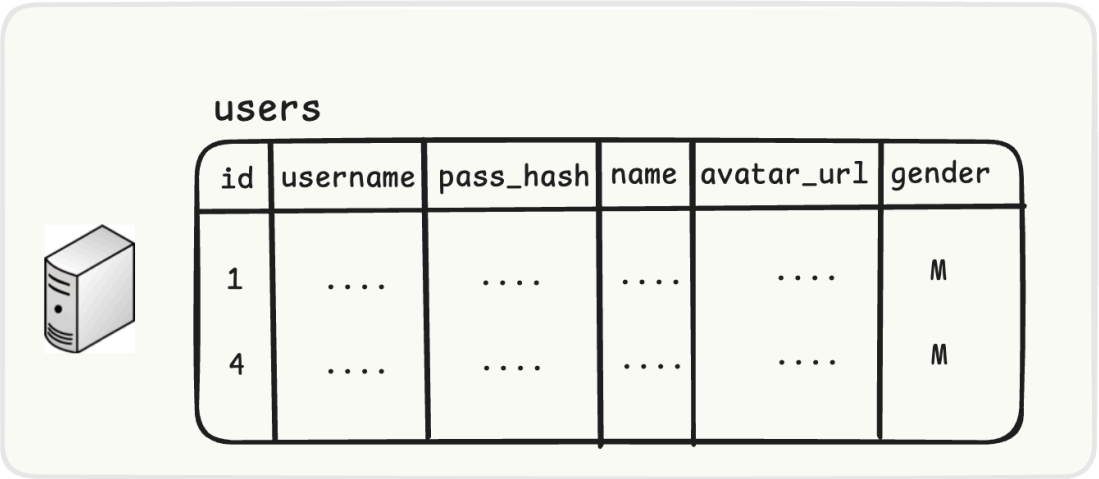
why?
Migrate to microservice / separation of concerns

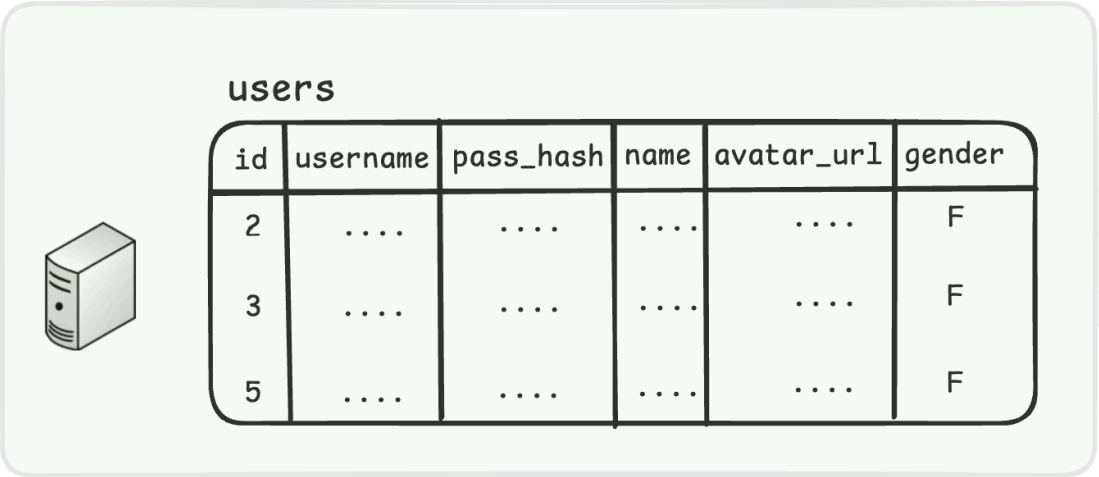



# **Horizontal Partitioning** (across servers)

why?
All the data cannot fit on a single server
*(note that there are other benefits of sharding — improves both read & write throughput)*




# Sharding

Sharding is simply "Horizontal Partitioning across servers"

### Sharding Key

Sharding is always based on some column - sharding key
*How to choose a good sharding key - [we will see this in a later lecture]*

### Q: Can we have multiple sharding keys for the same table?

No. You can only have one sharding key for any piece of data.
The chosen sharding key can be composite `(city, gender)` or `user_id`, ...

# Sharding ⇔ Routing

It should not be the case that sharding following `logic A`, but routing follows `logic B`.
Why?
Because if that is the case - then the requests will end up on servers that don't contain the necessary data!

*The logic / algorithm must be the same for both sharding & routing*

Therefore, all we have to do is use a Routing Algorithm. There's no "dedicated" sharding algorithm.

Sharding happens via Routing

Routing is the thing that happens - sharding is just a side effect of routing!
If we're routing based on user-id, then automatically the sharding will happen based on user-id
*Because if Sanjana's requests are being routed to server A, then only server A will be able to the store Sanjana's data (because server B / C never received requests with Sanjana's data in the first place!)*

We only decide `(which request -> which server)`
The moment we decide `(which request -> which server)`,
we've also automatically decided `(which data -> which server)`

So we choose a sharding key, and then we will use that sharding key for routing.

### Q: If all of Mukesh's data lives in exactly 1 server (determined by sharding key), then isn't that server a SPoF for Mukesh?

Absolutely True!
To fix this, we will do "Replication" *(discussed later)*

# Routing Algorithms

Routing algorithm runs inside the Load Balancers.
It is how the LB decides which request goes to which server

## How "Good" is your routing?

What characteristics should a good routing algorithm have?

- **Fast -** computationally easy to decide which request goes to which server

- **Equal distribution -** should divide the load (data & requests) equally across all servers
    - it should not be the case that some server is getting 2x or 3x more load than other servers
- **We should be able to freely add/remove servers**
    - servers are unreliable, so they can crash, so the number of servers can decrease
    - we might need to scale out more, so we might add more servers, so the number of servers will increase
- **Minimal data movement -** When the number of servers change, the data movement must be minimal (because data movement is expensive)
    - imagine that you're adding a new server
    - currently this new server doesn't contain any data
    - thanks to the routing algo, some of the data & load will be "shared"/"distributed" to this server
    - (because this server should share the load equally)
    - which mean that we need to copy some data to this server
    - this means that there's data movement
- **Routing should be deterministic without exchanging information**
    - we've multiple load balancers
    - all the load balancers should always be "in-sync"
        i. they should forward the same type of requests (say Sanjana's requests) to the same server because otherwise, Sanjana's data will be all over the place
        ii. it should not the case that different LBs send the same user to different servers
    - They must not have to constantly communicate with each other to be "in-sync" - because constant communication for every request will be too much of an overhead

# Round Robin

Send the next request to the next server.

Simple **%** based technique

```
// LB will use health-check / heartbeat to
// maintain this list of servers
server_list = ["10.11.6.12", "10.11.5.17", ...]

fn handle_request(request):
    key = request.user_id   // sharding key
    N = len(server_list)
    server_id = key % N
```

```
    server = server_list[server_id]
    forward_request(request, server)
```

### Example

```
server_list = [A, B, C, D]
user_id = 0 .... 100
```

### Initial Distribution

```
A   0   4   8     ...
B   1   5   9     ...
C   2   6   10    ...
D   3   7   11    ...
```

*n = 4*

### Adding / Removing servers

let's say server B crashes.

~~server_list = [A, B, C, D]~~

~~N = 4~~

server_list = [A, C, D]

N = 3

```
A   0   3   6   9     ...
B   1   5   9         ...
C   1   4   7   10    ...
D   2   5   8   11    ...
```

Since server B has crashed, and users (1, 5, 9, ..) were previously on server B, of course, their data has to be migrated

However, other users' data should not have to be migrated needlessly because their servers are still working - there's no need to move their data!

Is that the case?
User 2 was earlier going to server C. And even though Server C is still running, User 2 was needlessly migrated to server D.

No! Pretty all the data gets shuffles around.

Similarly, when we add servers, once again, the value of *N* will change. The value of *(user_id % N)* will also change for pretty much everyone.

### Pros

Fast, Equal Distribution, No need of syncing information

### Cons

Lots of unnecessary data movement!

### When can we use Round Robin?

It's a good algorithm, but, it causes unnecessary data movement when the number of servers change.
We can use Round Robin only when the number of server is static.
(ElasticSearch uses Round Robin)

# ~~Bucketting~~

Assign the `server → user_id` in ranges

```
server_list = [A, B, C, D]
total_users = 400

def handle_request(req):
    key = req.user_id
    N = len(server_list)
    bucket_size = total_users / N
    server_id = Math.floor(key / bucket_size)
```

server A will get user_ids `0 ... 99`
server B will get user_ids `100 ... 199`
server C will get user_ids `200 ... 299`
server D will get user_ids `300 ... 399`

total users = `400`
total servers = `4`
users / server = `400/4 = 100`

### Adding / Removing servers

if Server B crashes

total users = `400`
total servers = `3`
users / server = `400/3 = 133`

A will get user_ids `0 ... 132`
C will get user_ids `133 ... 265`
D will get user_ids `266 ... 399`

All the buckets will change, so there will be lots of unnecessary data movement.

### More users sign up

We cannot accommodate new users in existing servers - because we have already decided the buckets.
So it is impossible to add new users without first buying more servers

## Pros

Equal data distribution
Fast

## Cons

Too much data re-shuffling
Cannot even add users

# ~~Mapping Table~~

What if instead of having a formula based solution `(%, /, …)` we simply keep track of which user is in which server?

What if the LB maintains a Hashmap from `user_id` to `server_id`?

```
server_list = ["10.11.6.12", "10.11.5.17", ...]
mapping = {
    sanjana: 0
    prem: 1
    bathula: 0
    pallavi: 3
    venkat: 2
```

```
    ...
}

fn handle_request(request):
    key = request.user_name
    server = mapping[key]
    forward_request(request, server)
```

### Adding / Removing servers / users

Let's suppose server B crashes

```
fn handle_server_crash(crashed_server):
    for user, server in mapping:
        if server == crashed_server:
             // assign a new server to this user
             new_server = get_random_server()
             mapping[user] = new_server
             son_pari_please_migrate(user, crashed_server,
new_server)

fn add_server(new_server):
    // take some of the load of existing servers
    total_users = len(mapping)
    total_servers = len(set(mapping.values())) + 1
    number_of_users_to_move = (total_users // total_servers)
    // now simply go to these many random users
    // and assign them to this new server

    all_users = mapping.keys()
    chosen_users = random.choice(all_users, number_of_users_to_move)

    for user in chosen_users:
        mapping[user] = new_server
        son_pari.request_data_movement(...)


fn handle_new_user(user):
     server = get_random_server()
     mapping[user] = server
```

If we do this, will the data of people that were earlier on the crashed server get moved? Yes (that is desired)
For the users whose servers are still running, will their data get moved? No.

### Pros

Equal data distribution, Fast, *minimizes data movement*

### Cons

We will have to keep this mapping table in sync for all the LBs.
If different LBs have a different mapping table, then the requests will end up in random servers!

Keeping data in sync, always, with very low latency → extremely hard problem **IMPOSSIBLE !**
CAP Theorem — you cannot achieve perfect sync with low latency.

# Consistent Hashing

### Pros

*All of the above!*

- Fast
- equal load distribution
- freely add/remove servers
- minimal data movement
- no need for any sync between LBs

### Cons

*None!*

# Hash Function

```
fn add(a, b):
    return a + b
```

Function is a “deterministic” mapping from inputs to outputs.
add(2, 3)  => 5

add(2, 3)  => 5
add(2, 3)  => 5
*no matter how many times I call a function, it will always give me the same result for the same input*

```
state = 0
~~fn~~ proc add(a, b):
    state = b
    return a + b + state
```

in this case, the ~~function~~ procedure (impure function) doesn't always return the same value - because it has side effects

Hash function is a "digest" function — a function that takes as input anything, but returns values within a fixed range.

Inputs are unbounded, but the outputs are bounded within a range.

```
fn hash_1(data):
    # takes any string as input, and returns values
    # strictly b/w [0, 99]
    return (sum of ascii values of chars of data) % 100
```

in this case, no matter what the data, the output will always be from 0 .. 99

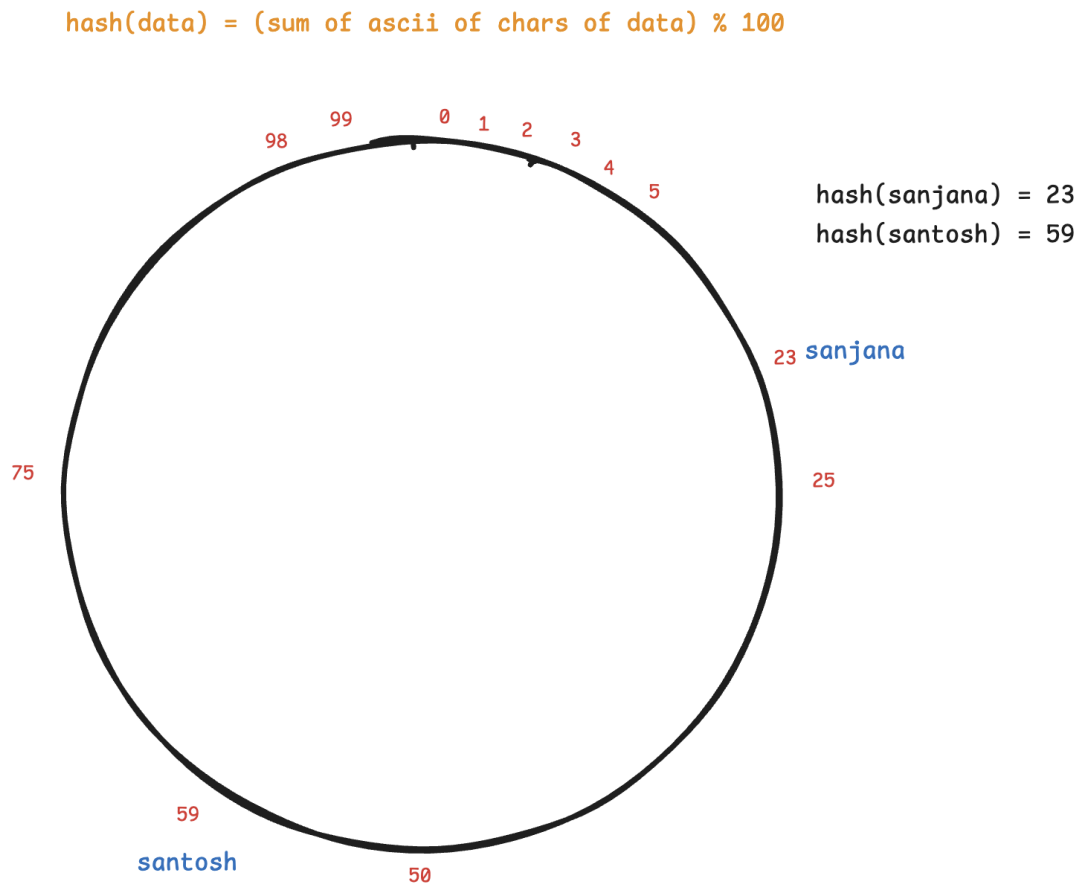
```
fn hash_2(data):
    return (sum of squares ascii values of chars of data) % 799

fn hash_3(data):
    return (multiply even and odd numbers in the data) % 1337
```

## **Resources on Hash Functions** (optional)

- **Definitions and properties:** Hash function - Wikipedia
- **Cryptographically secure (extra guarantees):** Cryptographic hash function - Wikipedia
- **Family of hashes:** Universal hashing - Wikipedia

## Hash Ring - output space of a given hash function

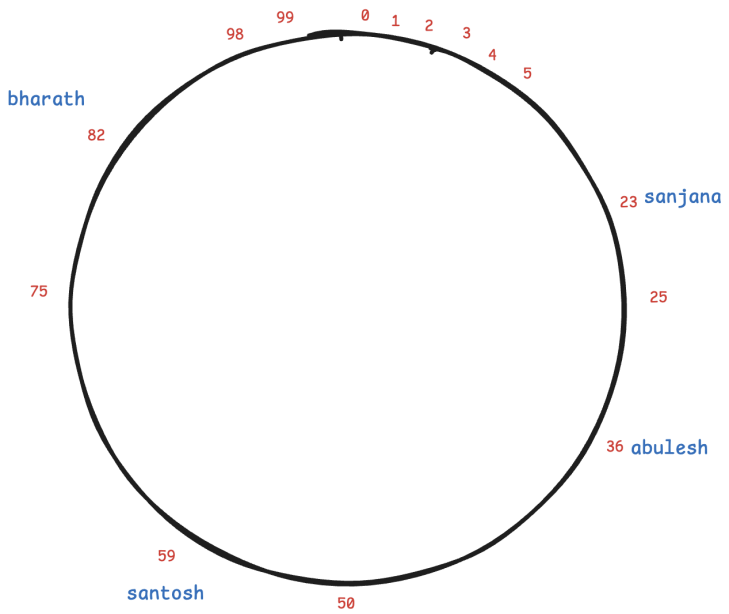



# Placing Users & Servers on the Hash Ring

https://en.wikipedia.org/wiki/Pseudorandomness

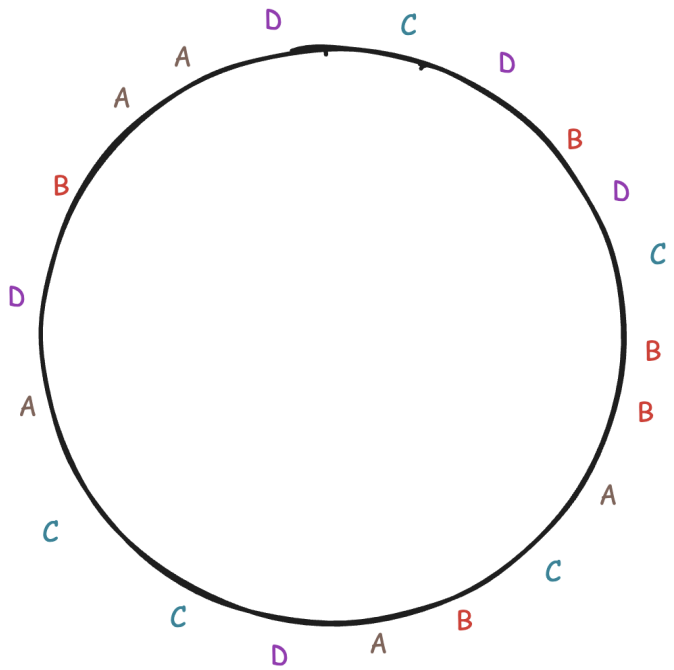
1. Consider 1 hash function `hash_0` for hashing the users

hash(data) = (sum of ascii of chars of data) % 100




hash_0(sanjana) = 23
hash_0(santosh) = 59
hash_0(abulesh) = 36
hash_0(bharath) = 82

2. Consider k hash functions hash_1 ... hash_k for hashing the servers
**typical value of k is 32 or 64**




Servers: A, B, C, D
k = 5 hashes for servers

hash_1(A) = 35
hash_2(A) = 49
hash_3(A) = 70
hash_4(A) = 89
hash_5(A) = 91

hash_1(B) = 43
hash_2(B) = 29
hash_3(B) = 30
hash_4(B) = 85
hash_5(B) = 12

hash_1(C) = ...
...

hash_1(D) = ...
...

**These hash functions are set-up while the LB is being coded/configured.**
**All the LBs for a system will have the same set of hash functions in their code.**

All these (k+1) hash functions have the same output space (say $0 \ldots 2^{64} - 1$)
https://en.wikipedia.org/wiki/K-independent_hashing

### Can there be hash collisions?

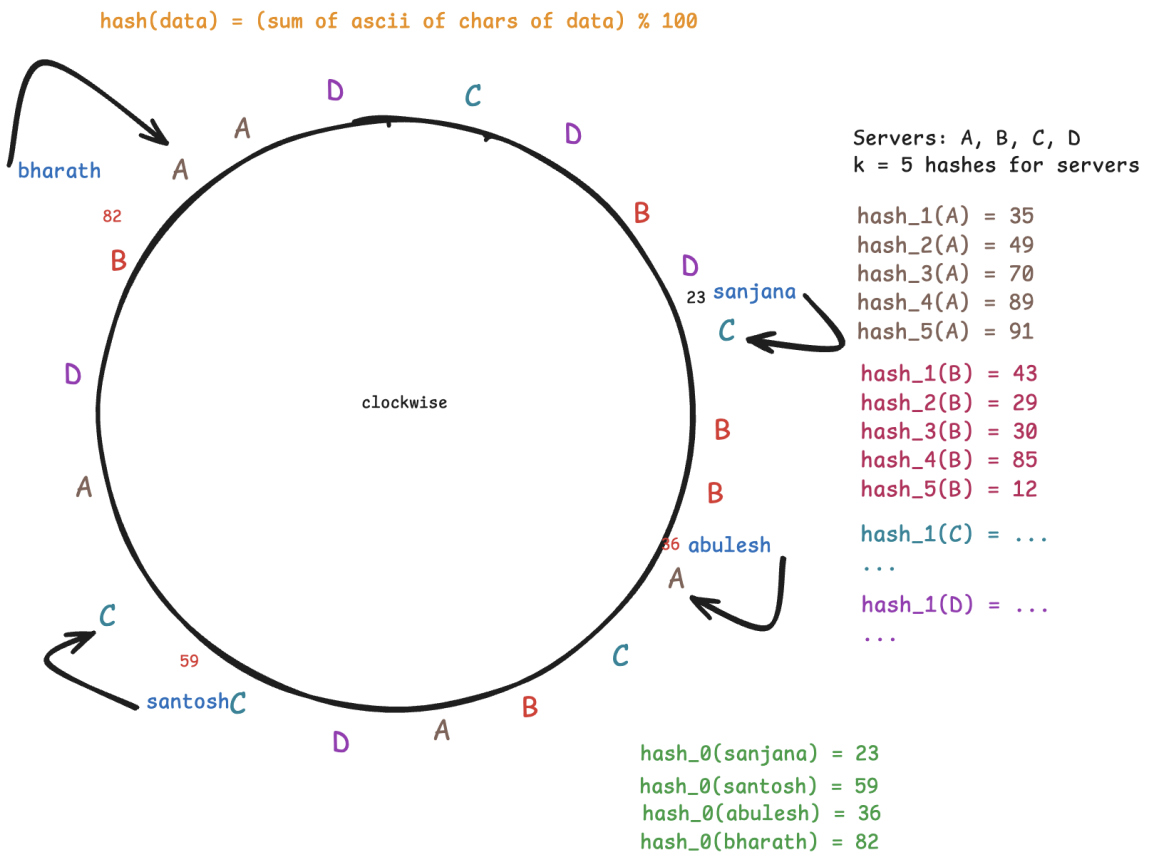
Yes, but they're unlikely.

Suppose our hash function outputs values in the range
`0 ... (2^64 - 1)`
Given 1 billion users, and 1 million servers, what is the probability that two hashes collide?
This probability will be very close to 0.

Even if a hash collision happens, that won't effect our algorithm at all.

# Consistent Hashing Algorithm

1. For each server that we have, we will hash it using all the hash functions `hash_1 .. hash_k` and put the server on the ring..
2. Whenever a request arrives, we will hash the `user_id` using the `hash_0` and place the user on the ring
3. The request will be forwarded to the first server on the ring which is in the clockwise direction (to the right of the user)



```python
import bisect

def custom_hash(data, i):
    """ i acts as the hash_function_number """
    h = sum(ord(c) ** i | 0xaff for c in data)
    h *= 3
    h >>= 1
    return h % 10

def build_ring(servers, k):
    ring = []
    for server in servers:
        for i in range(1, k+1):  # 1..k
            h = custom_hash(server, i)
            print(f'adding {server} to spot {h}')
            ring.append((h, server))

    ring = sorted(ring)
    print('final ring:', ring)
    return ring

def route(user):
    user_hash = custom_hash(user, 0)

    # I now need to find the 1st server in the ring
    # that is to the right of the user
    index = bisect.bisect_right(ring, (user_hash, user))
    index = index % len(ring)  # wrap it around the ring
    server_hash, server = ring[index]
    print(f'{user} hashed to {user_hash}. Routing them to {server} at location
{server_hash}')
    return server

servers = [
    '10.11.1.13',
    '10.11.2.17',
    '10.11.13.167',
    '10.11.12.255',
    '10.11.1.0',
]
ring = build_ring(servers, 3)

route('Vishal')
route('Vishal')
route('Sanjana')
route('Vishal')
route('Sanjana')
route('Sanjana')
```

### Is the algo deterministic?

will Sanjana always go to server C (as long as Sanjana is alive, and server C is running, and not servers are added/removed)?
**Yes!**

### Is the algo fast?

Yes! The ring is a sorted array, and to find the nearest server in the clockwise direction, you have to do a binary search (upperbound) for the user's hash.
$O(\log_2(Nk))$ where N is the number of servers, and k is the number of server hashes

Google has 10 million servers, and let's suppose they use k=64,
then how many iterations of binary search?

$\log_2$ (64 * 10 million)
= $\log_2$ (64) + $\log_2$(10) + $\log_2$(1 million)
= 6 + 3.xyz + 19.xyz
= approx 30

This will take 3 micro seconds.

### Do the LBs have to share any data to be in sync?

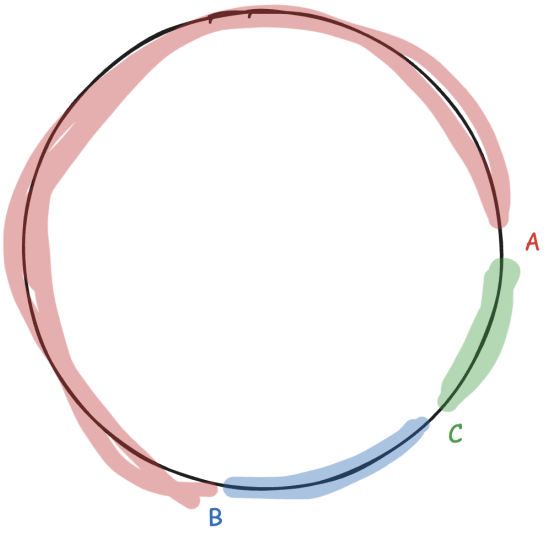
No!

- Each LB knows which servers are running (via health-check/heartbeat)
- The code in each LB is the exact same
- So, each LB will calculate the same hashes and the same ring
- So, each LB will calculate the same server for any user

Without the need for explicit syncing, the LB will agree upon the routing!

### Is the data distribution even?

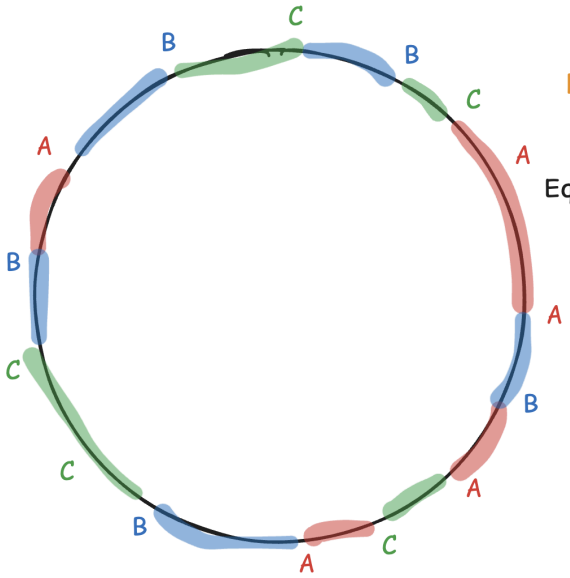
Yes. As the value of k increases, the distribution becomes more and more even




k = 1

A, B, C

unequal distribution
A is overloaded




k = 5

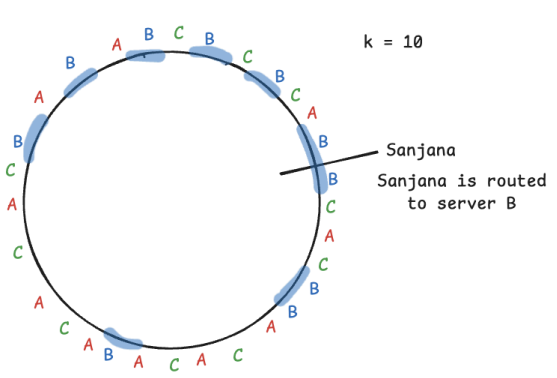
Equal distribution!

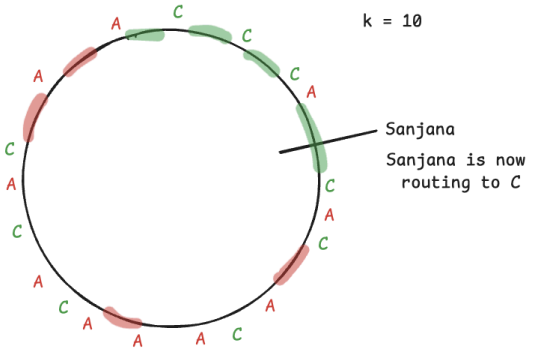
# Adding / Removing servers

When a server goes down

1. the users of this server are automatically redirected to other servers
2. the load of the downed server is automatically redistributed equally across the rest of the servers
    a. prevents cascading failure
        i. cascading failure happens when all the load of the downed server goes to very few servers
        ii. the servers which now handle additional load get overloaded and go down as well
        iii. which causes a chain reaction
        *iv. cascading failure*

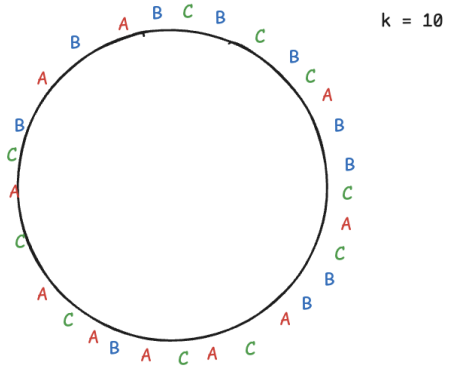
3. Any users that were going to uncrashed servers continue to do so
    a. No unnecessary data movement
    b. only the users whose server has crashed get redirected
    c. the data has to be moved for only those users

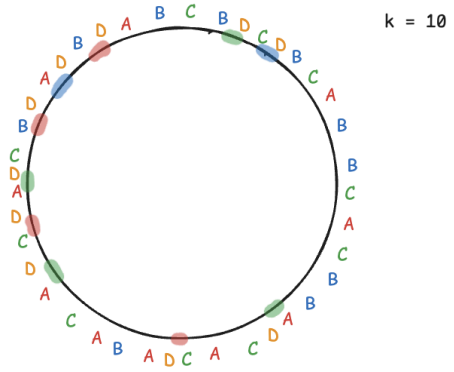



When a new server is added, it takes up some of the load of every other server

1. only some of the users from each server will be moved to the new server




# Data Movement

### When a server dies, how do we ensure that the user's data is not lost?

**Data Replication** — every piece of data will be stored multiple times in our system (each copy will be in a different server)

### When a new server gets added, we need to move some data from existing servers to the new server - with minimal downtime. How?

We will assume that there's a magic fairy that moves the data flawlessly and magically for us

We will dissect Son Pari in lecture 8.

# Stateless Servers

**State = Data**
Stateful = stores data
Stateless = doesn't store data

# Issues with current design

Current design stores both application logic and data in the same servers.
Each server acts as both the app server and the db server.

1. We cannot scale things independently
    a. App server - powerful CPU, RAM, good network to handle the requests
    b. Database server - powerful CPU, RAM, lot of HDD to handle the queries & store data

    since both these things are happening in all server, all servers have to be very powerful
2. When the app server is updating, the data is not available
    a. any data on server S can only be accessed by the application code on server S
    b. when the app code on server S is being updated, the data on server S is not available

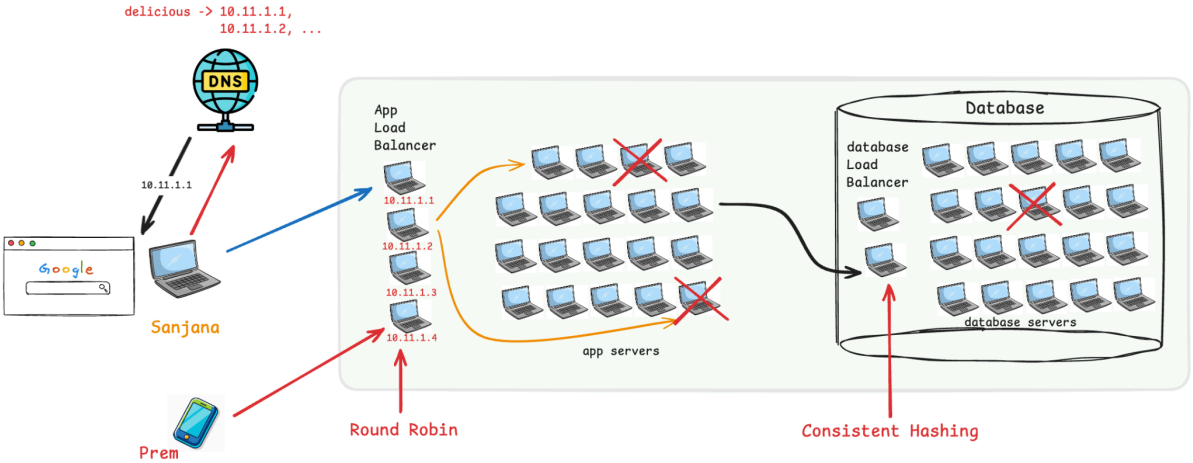
    increased downtime
    Because of CI/CD any any given time some server will always be updating
    (microsoft does ~2000 deployments every single day)

3. the developer has to worry about how exactly sharding is happening
   app developer also has to be an expert database developer (they must know the internals of databases)

# Improved Design

The database and application layers have been separated

1. The app developer sees the data as a single entity
   a. they don't care how the DB is sharded
2. The database has its own Load Balancer
3. Application servers now don't store any data
   App servers are now **Stateless**
4. The app load balancer now can simply use Round Robin
5. Consistent hashing is implemented internally by the DB load balancer (the app developer doesn't have to worry about it)




```
db_url=postgres://10.11.0.1:5302/maindb;postgres://10.11.2.3:5302/maindb;...
```

# Readings

### Mandatory

Load Balancer

### Optional

- **How to configure:** https://docs.google.com/document/d/1DxQzLpu1XPe_mRWsewNWtKL6E4uwKHQhBp7GX6Sg7qI/edit#heading=h.fzwcs020kou5
- **Tutorial:** https://avinetworks.com/what-is-load-balancing/
- **Elastic Load Balancer (ELB - AWS):** https://aws.amazon.com/elasticloadbalancing/getting-started/

# Doubts

`142.250.192.4`

**IPv4:** `[0-255] . [0-255 ] . [0-255] . [0-255]`
**total number of possibilities:** $2^8 * 2^8 * 2^8 * 2^8 = 2^{32}$ ~ 4 billion
**number of devices on the internet**: 100 billion to 1 trillion
*Static vs Dynamic IPs*

**IPv6:** 128 bits
**Number of possibilities:** $2^{128}$ ~ 256 billion billion billion billion *(sufficient even if we colonize the entire observable universe!)*